

Equations	Coefficients			Roots		Predicted roots (neural network outputs)	
	a	b	c	r1	r2	pre-r1	pre-r2
1	1	5	6	-2	-3.0	-1.9999879917	-3.000010276
2	1	-2	-8	4	-2.0	3.2248691572	-3.467243981
3	2	1	-3	1	-1.5	0.9677422405	-1.716275669

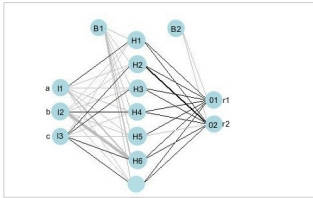
Table 1: Outputs of `print()` in tabular form

Figure 1: Neural network model of the quadratic equation solver

```
net.quadroot <- neuralnet(r1+r2~a+b+c, trainingdata,
hidden=7, threshold=0.0001)
```

An arbitrary performance threshold value 10^{-4} is taken and this can be adjusted as per requirements.

The configuration of the just-constructed model with its training data can be visualised with a `plot` function.

```
#Plot the neural network
plot(net.quadroot)
```

Now it is time to verify the neural net with a set of unknown data. Arbitrarily take a few sets of values of three coefficients for the corresponding quadratic equations, and arrange them as data frames as shown below.

```
x1<-c(1, 1, 1, -2, 1, 2)
x2<-c(5, 4, -2, -1, 9, 1)
x3<-c(6, 5, -8, -2, 22, -3)
```

Since there is no zero coefficient corresponding to x^2 , we can accept only those coefficients for which the discriminant is zero or more than zero.

```
disc <- (x2*x2-4*x1*x3)
k <- which(disc >= 0)
x1 <- x1[k]
x2 <- x2[k]
x3 <- x3[k]
y1=as.data.frame(x1)
y2=as.data.frame(x2)
y3=as.data.frame(x3)
```

The values are then fed to the just-configured neural model `net.quadroot` to predict their roots. The predicted roots are collected into `net.result$net.result` and can be displayed with the `print()` function.

```
testdata <- cbind(y1,y2,y3)
net.results <- compute(net.quadroot, testdata)
#Lets see the results
print(net.results$net.result)
```

Now, how does one verify the results? To do this, let us compute the roots using the conventional root calculation formula, and verify the results by comparing the predicted values with them.

Calculate the roots and concatenate them into a data frame.

```
calr1 <- (-y2 + sqrt(y2*y2-4*y1*y3))/(2*y1)
calr2 <- (-y2 - sqrt(y2*y2-4*y1*y3))/(2*y1)
r<-cbind(calr1,calr2) #Calculated roots using formula
```

Then combine the test data, its roots and the predicted roots into a data frame for a decent tabular display for verification (Table 1).

```
#Combine Inputs, Expected roots and predicted roots.
comboutput <- cbind(testdata,r,net.results$net.result)
#Put some appropriate column heading
colnames(comboutput) <- c("a","b","c","r1","r2","pre-
r1","pre-r2")
print(comboutput)
```

It is clear from the above outputs that our neural network has learnt appropriately and produces an almost correct result. You may need to run the neural network several times with the given parameters to achieve a correct result. But if you are lucky, you may get the right result in the first attempt itself! **END** 

By: Dipankar Ray

The author is a member of IEEE, IET, with more than 20 years of experience in open source versions of UNIX operating systems and Sun Solaris. He is presently working on data analysis and machine learning using a neural network and different statistical tools. He has also jointly authored a textbook called 'MATLAB for Engineering and Science'. He can be reached at dipankarray@ieee.org.